

Multi threading in C++ (C++ >= 11)

Indice

Primitives	1
Threads	1
Mutexes	2
Condition variables	3
Design patterns	4
Producer/consumer	4
Active objects	5
Reactor	6
Thread pool	7

Primitives

Threads

To create a new thread we instantiate the thread class (found in <thread>). Some member functions are:

1. `get_id()`
2. `detach()`: allows a thread to run independently from the others
3. `join()`: blocks waiting for the thread to complete
 - All threads are created joinable
 - The data structure used by joinable threads is deallocated only if the thread is join-ed
4. `joinable()`
5. `hardware_concurrency()`: a hint on the hardware thread contexts (basically how many CPU cores we have available)
6. the `=` operator to move (thread is movable, but not copyable)

The standard also provides the `std::this_thread` sub-namespace to group a set of functions used to access the current thread.

- `get_id()`
- `yield()`: suspend the current thread
- `sleep_for()`: sleep for a certain amount of time
- `sleep_until()`: sleep until a given absolute timestamp

The thread object constructor can take additional arguments to pass to the thread function. Obviously these arguments need to be matched by the function we use for the thread

```
##include <iostream>
##include <thread>

using namespace std;
using namespace std::chrono;

void myThread(const string& s) {
    for(;;) {
```

```

    cout << s << endl;
    this_thread::sleep_for(milliseconds(500)); // we can also use 500ms since
                                              // in cpp we have user defined
                                              // literals and units
}
}
int main() {
    thread t(myThread, "world ");
    myThread("hello");
    t.join();
}

```

Mutexes

C++ has the mutex class (found in <mutex>). The main functions are:

1. `lock()`: lock the mutex, blocking if it is already lock
2. `try_lock()`: like `lock()`, but do not block
3. `unlock()`: unlock the mutex

Using `lock` and `unlock` manually is still prone to programming errors (e.g. for some reason the function could exit without unlocking the mutex), leading to deadlocks or incomplete locking. C++ provides scoped locks that automatically unlock the mutex after we exit the scope:

1. `lock_guard<>`: the simplest variant, works like we would expect it: locks on object creation, unlocks on object deallocation.

```

// ...
mutex myMutex;
int sharedVariable;
void myFunction(int value) {
    { // create new scope; variables declared in the scope get deallocated at
      // the end of it
      lock_guard<mutex> lck(myMutex);
      if (value<0) {
          cout<<"Error"<<endl;
          return;
      }
      SharedVariable += value;
    }
}

```

2. `unique_lock<>`: includes the `lock_guard` features but extend its functionalities with the concept of ownership
 - The ownership of a mutex can be moved from one `unique_lock` object to another
 - The mutex lock can be deferred with respect to the object construction (construct passing `defer_lock` and `lock()` at the starting point)
 - The mutex unlock can be explicitly performed before the object destruction
 - Has more overhead compare to `lock_guard<>`

These classes implement the RAII pattern, wrapping the lower-level mutex class.

`recursive_mutex` solves a simple problem: multiple locks by the same owner (due e.g. to multiple functions that all lock the same mutex). `recursive_mutex` allow the owner of it to lock it any number of time; to unlock it we need to unlock it the same number of times we locked it. `recursive_mutex` has higher overhead than a regular mutex.

A deadlock can be introduced by the incorrect order of locking/unlocking. C++ has a solution to this: the `lock` function can take many mutexes and it guarantees that these locks will always be locked/unlock in the same order. `lock` can be paired with `lock_guard<>` with `adopt_lock` to ensure we have scoped locks that are locked in the correct order.

```

mutex myMutex1;
mutex myMutex2;

```

```

void func2() {
    lock(myMutex1, myMutex2);
    lock_guard<mutex> lk1(myMutex1, adopt_lock);
    lock_guard<mutex> lk2(myMutex2, adopt_lock);
    doSomething2();
}

void func1(){
    lock(myMutex2, myMutex1);
    lock_guard<mutex> lk1(myMutex1, adopt_lock);
    lock_guard<mutex> lk2(myMutex2, adopt_lock);
    doSomething1();
}

```

C++17: scoped_lock C++17 introduces the `scoped_lock` class that basically does what `lock` + `lock_guard` do.

C++17: shared_lock To squeeze out as much performance as possible, we need to keep the critical section to the absolute minimum, without introducing wrong accesses. As long as critical sections only read shared variables, they don't interfere (the variables can be copied to a local copy in the critical section). This is the Read-Write lock paradigm:

- Multiple readers are allowed to access the resource as long as no one is writing
- A single writer at a time is allowed to access the resource

This type of lock, implemented by `shared_mutex` in C++, can be locked in two ways:

1. Shared (`lock_shared`): locked for reading, ownership is shared among readers
2. Exclusive (`lock`): locked for writing, ownership in of only one writer

The `shared_lock` class (found in `<shared_lock>`) implements the RAII version of locking/unlocking in shared mode. To lock a `shared_mutex` in exclusive mode one should use `unique_lock`.

Condition variables

In multi-threaded programs we may have dependencies among threads. A dependency can come from the fact that a thread must wait for another one to complete its current operation. Differently from locking a mutex, where waiting is undesirable but necessary, this waiting must be completely inexpensive.

To implement this we need a way to explicitly block a thread, put it into a waiting queue and notify it when the condition leading to the block has changed.

The `condition_variable` class implements the following methods:

1. `wait(unique_lock<mutex> &):` blocks the thread until another thread wakes it up; the Lockable object is unlocked for the duration of the call
2. `wait_for():` blocks the thread until another thread wakes it up, or a time span has passed
3. `notify_one():` wake up one of the waiting threads
4. `notify_all():` wake up all the waiting threads; if no thread is waiting do nothing

Typical usage for condition variables:

```

#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>

using namespace std;

static string shared;
static mutex myMutex;
static condition_variable myCv;

void myThread() {

```

```

string s;
{
    unique_lock<mutex> lck(myMutex);
    while(shared.empty())
        myCv.wait(lck);
    s = shared;
}
cout << s << endl;
}

```

```

int main() {
    thread t(myThread);
    string s;

    cin >> s; // read from stdin
    {
        unique_lock<mutex> lck(myMutex);
        shared = s;
        myCv.notify_one();
    }
    t.join();
}

```

condition_variable_any is a generalization of condition_variable that can also be used with any Lockable type (this includes shared_lock). It has more overhead than the basic condition_variable.

Design patterns

Design patterns are reusable solutions to common problems that allow to avoid common pitfalls.

Producer/consumer

A thread (consumer) needs data from another thread (producer). To decouple the operations of the two threads we put a queue between them. The access to the queue needs to be synchronized not only using a mutex because the consumer needs to wait if the queue is empty; optionally, the producer may block if the queue is full.

Example of a producer/consumer with a unlimited size queue.

```

// ##### synchronized_queue.h
#ifndef SYNC_QUEUE_H_
#define SYNC_QUEUE_H_
#include <list>
#include <mutex>
#include <condition_variable>

template<typename T>
class SynchronizedQueue {
public:
    SynchronizedQueue(){}
    void put(const T & data); // we can also define move semantics if we want
    T get();
    size_t size() const;
private:
    SynchronizedQueue(const SynchronizedQueue &)=delete;
    SynchronizedQueue & operator=(const SynchronizedQueue &)=delete;
    std::list<T> queue;
    mutable std::mutex myMutex;
}

```

```

    std::condition_variable myCv;
};

template<typename T>
void SynchronizedQueue<T>::put(const T& data) {
    std::unique_lock<std::mutex> lck(myMutex);
    queue.push_back(data);
    myCv.notify_one();
}

template<typename T>
T SynchronizedQueue<T>::get() {
    std::unique_lock<std::mutex> lck(myMutex);
    while(queue.empty())
        myCv.wait(lck);
    T result=queue.front();
    queue.pop_front();
    return result;
}

template<typename T>
size_t SynchronizedQueue<T>::size() {
    std::unique_lock<std::mutex> lck(myMutex);
    return queue.size();
}
#endif

// ##### main.cpp
#include <iostream>
#include <thread>

#include "synchronized_queue.h"

using namespace std;
using namespace std::chrono;

SynchronizedQueue<int> queue;

void myThread() {
    for(;;) cout<<queue.get()<<endl;
}

int main() {
    thread t(myThread);
    for(int i=0; ; i++) {
        queue.put(i);
        this_thread::sleep_for(seconds(1));
    }
}

```

Active objects

This patterns is used to instantiate “task objects”. A thread function has no explicit way for other threads to communicate with it; with this pattern we wrap the thread in an objects, allowing us to define methods for it.

```

// ##### active_object.h
#ifndef ACTIVE_OBJ_H_
#define ACTIVE_OBJ_H_
#include <atomic>
#include <thread>

```

```

class ActiveObject {
public:
    ActiveObject();
    virtual ~ActiveObject();
private:
    virtual void run()=0;
    ActiveObject(const ActiveObject &)=delete;
    ActiveObject& operator=(const ActiveObject &)=delete;
protected:
    std::atomic<bool> quit;
    std::thread t;
};
#endif // ACTIVE_OBJ_H_

// ##### active_object.cpp
#include "active_object.h"
#include <chrono>
#include <functional>
#include <iostream>

using namespace std;
using namespace std::chrono;

ActiveObject::ActiveObject(): quit(false),
                             t(&ActiveObject::run, this)
{}

ActiveObject::~ActiveObject() {
    if(quit.load()) return; //For derived classes
    quit.store(true);
    t.join();
}

```

The constructor initialize the thread object, while the destructor takes care of joining it. The run() member function acts as a “main” concurrently executing. The various task objects will extend ActiveObject, overriding the needed methods (run and the destructor).

Reactor

The Reactor class derives from ActiveObject to implement the executor thread and uses SynchronizedQueue for the task queue.

```

// ##### reactor.h
#ifndef REACTOR_H_
#define REACTOR_H_
#include <functional>

#include "synchronized_queue.h"
#include "active_object.h"

class Reactor: public ActiveObject {
public:
    void pushTask(std::function<void ()> func);
    virtual ~Reactor();
private:
    virtual void run();
    SynchronizedQueue<std::function<void ()>> tasks;
}

```

```

};
#endif // REACTOR_H_

// ##### reactor.cpp
#include "reactor.h"

using namespace std;

void doNothing() {}
void Reactor::pushTask(function<void ()> func) {
    tasks.put(func);
}
Reactor::~Reactor() {
    quit.store(true);
    pushTask(&doNothing);
    t.join(); // Thread derived from ActiveObject
}
void Reactor::run() {
    while(!quit.load())
        tasks.get(); // Get a function and call it
}

// ##### main.cpp
#include <iostream>
#include <chrono>

#include "reactor.h"

using namespace std;

void printAdd(int a, int b) {
    this_thread::sleep_for(seconds(5));
    cout<<a<<'+ '<<b<< '='<<a+b<<endl;
}

int main() {
    Reactor reac;
    int a, b;
    while(cin >> a >> b)
        reac.pushTask(bind(&printAdd,a,b));
}

```

Thread pool

The above implementation of Reactor executes task sequentially, meaning that the latency is proportional to the length of the queue. We could also have multiple executors picking tasks from a shared queue. This the basics of the thread pool pattern: we have one (or more) queue(s) of tasks/jobs and a fixed set of worker threads. The thread pool pattern allows to reduce thread creation overhead at the cost of two problems:

1. How many workers to use?
 - Usually a number somehow related to the number of available CPU cores
2. How to allocate tasks to threads?

```

// ##### threadpool.h
#ifndef THREADPOOL_H
#define THREADPOOL_H
#include <atomic>
#include <functional>

```

```

#include <mutex>
#include <thread>
#include <vector>

#include "synchronized_queue.h"

class ThreadPool {
private:
    std::atomic<bool> done;
    unsigned int thread_count;
    SynchronizedQueue<std::function<void()>> work_queue;
    std::vector<std::thread> threads;
    void worker_thread();
public:
    ThreadPool(int nr_threads = 0);
    virtual ~ThreadPool();
    void pushTask(std::function<void ()> func) {
        // SynchronizedQueue guarantees mutual exclusive access
        work_queue.put(func);
    }
    int getWorkQueueLength() {
        return work_queue.size();
    }
};

#endif // THREADPOOL_H

// ##### threadpool.cpp
#include "threadpool.h"

void doNothing() {}

ThreadPool::ThreadPool(int nr_threads): done(false) {
    if(nr_threads <= 0)
        thread_count = std::thread::hardware_concurrency();
    else
        thread_count = nr_threads;
    for(unsigned int i = 0; i < thread_count; ++i)
        threads.push_back(std::thread(&ThreadPool::worker_thread, this));
}

ThreadPool::~~ThreadPool() {
    done.store(true);
    for(unsigned int i = 0; i < thread_count; ++i)
        pushTask(&doNothing);
    for(auto & th: threads)
        th.join();
}

void ThreadPool::worker_thread() {
    while(!done)
        work_queue.get(); // Get a function and call it
}

```